Introdução

Sabrina de F. Souto.

Reflexão

- O que significa projetar um software?
  - É fácil?
    - Não
  - É rápido?
    - Não
  - É custoso?
    - Sim

Reflexão

- O que precisamos para garantir qualidade?
  - EXEMPLOS:
    - Trabalhar de forma colaborativa
    - REUSO

  - As pessoas enfrentam problemas similares
  - As pessoas resolvem esses problemas
  - Será que ter soluções próprias (novas) é viável?

Reuso

- 1960 – 1970
  - Reuso de linhas de código (*cut-and-paste*)
    - Bibliotecas: funções genéricas
- 1980 - 1990
  - Fase 1: Reuso de Classes (implementação)
    - Herança e Composição
  - Fase 2: Contrato de Relacionamento entre objetos
    - Interfaces (permite polimorfismo)
- Após 1990
  - Micro-arquiteturas que definem colaborações entre classes e objetos (Melhores idéias do mundo)
    - REUSO de IDEIAS
- Após 2000 ?

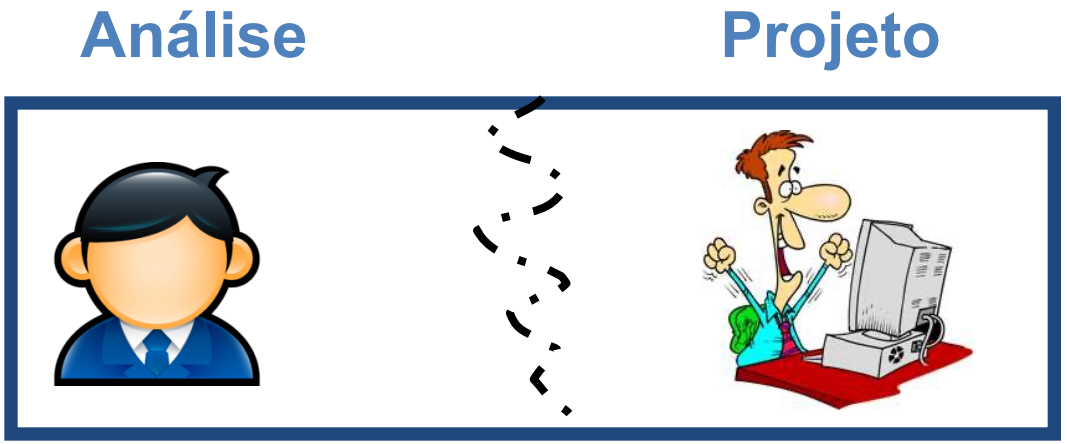
Reuso e Qualidade de Sw

- Padrões de Projeto
  - Reuso de *idéias*, não de código
    - Micro-arquiteturas de classes, objetos, seus papéis e suas colaborações
- Desenvolvimento baseado em componentes
  - Usa Componentes Pré-Fabricados
- Framework
  - Captura a funcionalidade comum a várias aplicações

O que queremos?

- O desenvolvimento de sistemas
  - Simples
  - Flexível
  - Reutilizável
- Nos preparamos para o desenvolvimento
  - Baseado em Componentes
  - De frameworks
- Análise e Projeto, Padrões de Projeto e UML

Análise e Projeto



- Análise (ponto de vista do cliente)
  - Entendimento do domínio do problema
  - Definição dos requisitos, use case, etc.
- Projeto (resultado para o programador)
  - Os elementos do domínio do problema são encapsulados em uma infra-estrutura técnica

Padrões de Projeto

- São de micro-arquiteturas de classes, objetos, seus papeis e suas colaborações
  - *"Isolar o que muda do que é igual"*

- "*Cada padrão descreve um problema que ocorre freqüentemente e então descreve o cerne da solução ao problema de forma a poder reusar a solução milhares de vezes em situações diferentes*"

Padrões de Projeto

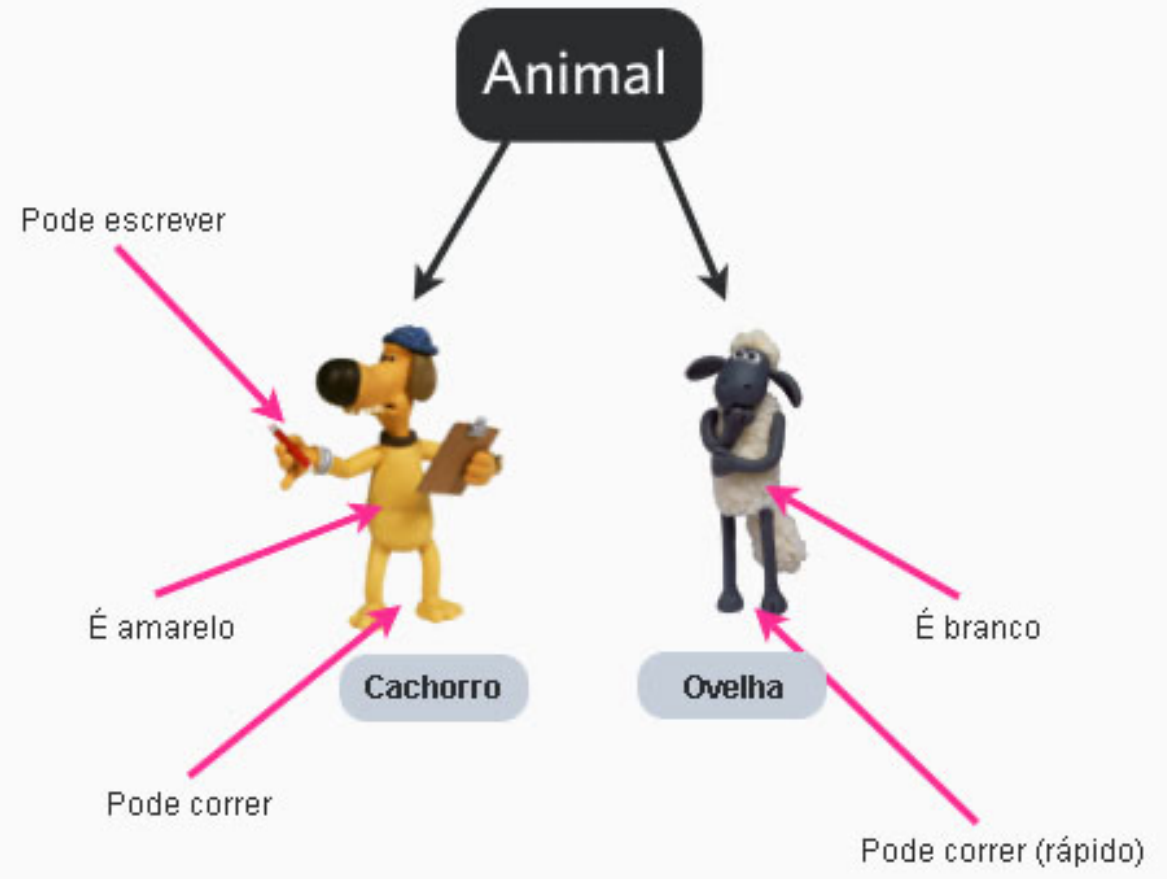
- **Um Nome**
  - Indicação inicial do problema do projeto, suas soluções e conseqüências
- **O Problema**
  - Problema e Contexto
  - Quando Aplicar
  - Condições para aplicação (as vezes)
- **A solução**
  - Estrutura (UML) com seus Participantes e Colaborações
  - Nível abstrato: *gabarito* de solução para várias situações
- **Conseqüências**
  - Perdas e ganhos com a aplicação do projeto

Padrões de Projeto

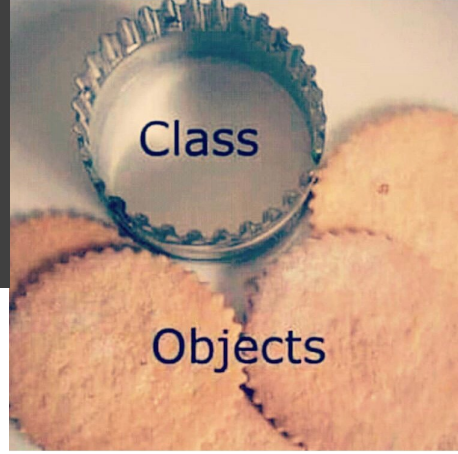
Básicos		Criação	Estruturais	Comportamento
Interface	Expert	Abstract Factory	Adapter	Chain of Responsibility
Polimorfismo	Creator	Builder	Bridge	Command
Herança	Baixo Acoplamento	Factory Method	Composite	Interpreter
Composição	Alta Coesão	Prototype	Decorator	Iterator
		Singleton	Façade	Mediator
			Flyweight	Memento
			Proxy	Observer
				State
				Strategy
				Template Method
				Visitor

Iremos aprender a...

- Pensar Orientado a Objetos
  - Buscar abstrações e soluções flexíveis



O que já devemos saber?



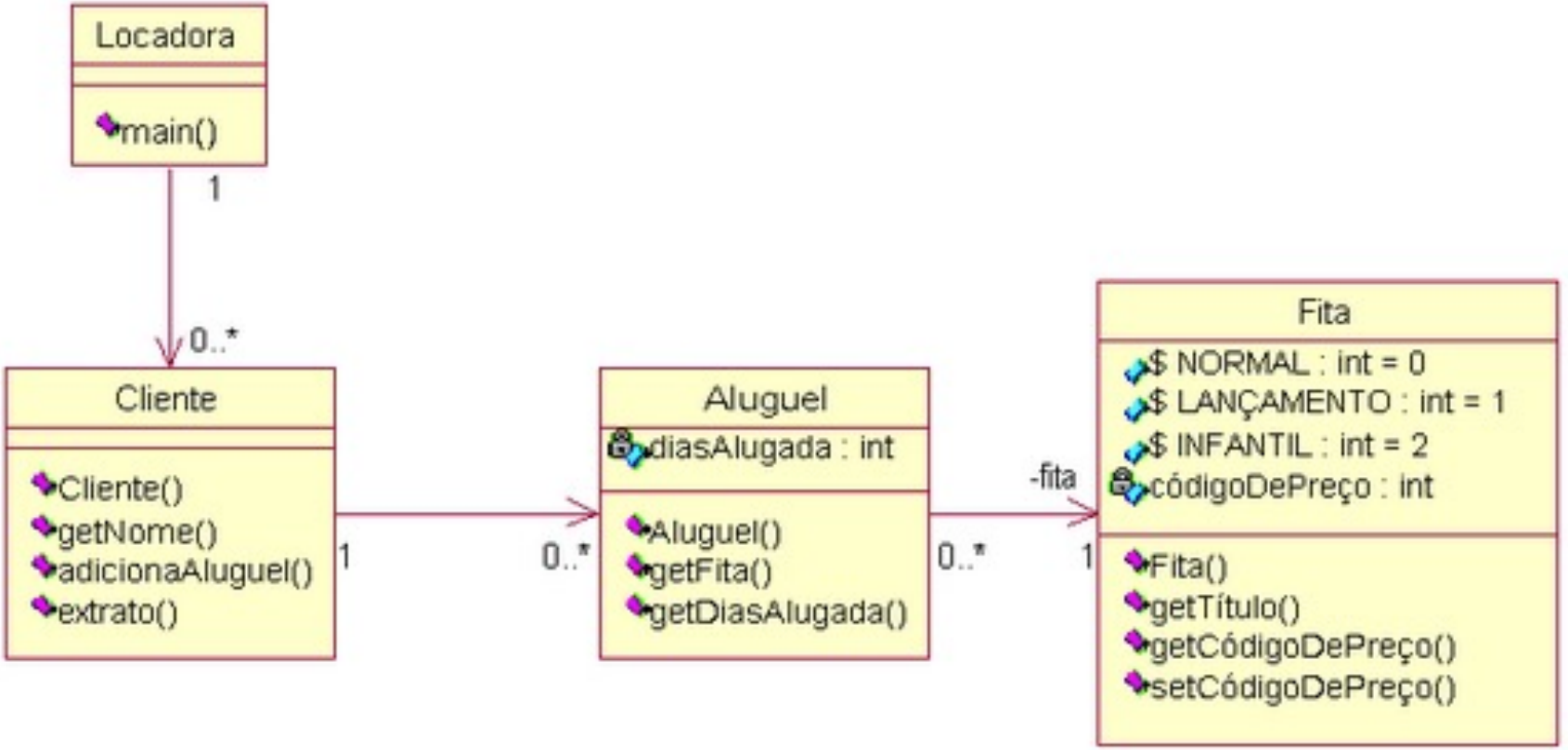
- O que são classes?
- O que são objetos?
- O que são propriedades?
- O que são responsabilidades?
- O que são os relacionamentos?
- Como os relacionamentos são definidos?

Programa Original

- Referência: Livro de Fowler: "*Refactoring*"
- O programa é simples: ele calcula e emite um extrato de um cliente numa vídeo-locadora
- Vamos executar o programa para ver uma saída possível:

```
C:\...\locadora-v1>java Locadora
Registro de Alugueis de Juliana
        O Exorcista                        3.5
        Men in Black                       2.0
        Jurassic Park III                  9.0
        Planeta dos Macacos                12.0
        Pateta no Planeta dos Macacos      12.0
        O Rei Leao                         42.0
Valor total devido: 80.5
Voce acumulou 8 pontos de alugador frequente
```

Diagrama de classes



```java
public class Fita {
  public static final int NORMAL = 0;
  public static final int LANÇAMENTO = 1;
  public static final int INFANTIL = 2;

  private String título;
  private int códigoDePreço;

  public Fita(String título, int códigoDePreço) {
    this.título = título;
    this.códigoDePreço = códigoDePreço;
  }

  public String getTítulo() {
    return título;
  }

  public int getCódigoDePreço() {
    return códigoDePreço;
  }

  public void setCódigoDePreço(int códigoDePreço) {
    this.códigoDePreço = códigoDePreço;
  }
}
```

```java
public class Aluguel {
  private Fita fita;
  private int diasAlugada;

  public Aluguel(Fita fita, int diasAlugada) {
    this.fita = fita;
    this.diasAlugada = diasAlugada;
  }

  public Fita getFita() {
    return fita;
  }

  public int getDiasAlugada() {
    return diasAlugada;
  }
}
```

```java
import java.util.*;

public class Cliente {
  private String nome;
  private Collection fitasAlugadas = new ArrayList();

  public Cliente(String nome) {
    this.nome = nome;
  }

  public String getNome() {
    return nome;
  }

  public void adicionaAluguel(Aluguel aluguel) {
    fitasAlugadas.add(aluguel);
  }
```

```java
  public String extrato() {
    final String fimDeLinha = System.getProperty("line.separator");
    double valorTotal = 0.0;
    int pontosDeAlugadorFrequente = 0;
    Iterator alugueis = fitasAlugadas.iterator();
    String resultado = "Registro de Alugueis de " + getNome() + fimDeLinha;
    while(alugueis.hasNext()) {
      double valorCorrente = 0.0;
      Aluguel cada = (Aluguel)alugueis.next();

      // determina valores para cada linha
      switch(cada.getFita().getCódigoDePreço()) {
      case Fita.NORMAL:
        valorCorrente += 2;
        if(cada.getDiasAlugada() > 2) {
          valorCorrente += (cada.getDiasAlugada() - 2) * 1.5;
        }
        break;
      case Fita.LANÇAMENTO:
        valorCorrente += cada.getDiasAlugada() * 3;
        break;
      case Fita.INFANTIL:
        valorCorrente += 1.5;
        if(cada.getDiasAlugada() > 3) {
          valorCorrente += (cada.getDiasAlugada() - 3) * 1.5;
        }
        break;
      } //switch
      // trata de pontos de alugador frequente
      pontosDeAlugadorFrequente++;
      // adiciona bonus para aluguel de um lançamento por pelo menos 2 dias
      if(cada.getFita().getCódigoDePreço() == Fita.LANÇAMENTO &&
         cada.getDiasAlugada() > 1) {
         pontosDeAlugadorFrequente++;
      }

      // mostra valores para este aluguel
      resultado += "\t" + cada.getFita().getTítulo() + "\t" + valorCorrente + fimDeLinha;
      valorTotal += valorCorrente;
    } // while
    // adiciona rodapé
    resultado += "Valor total devido: " + valorTotal + fimDeLinha;
    resultado += "Voce acumulou " + pontosDeAlugadorFrequente +
              " pontos de alugador frequente";
    return resultado;
  }
}
```

```
public class Locadora {
  public static void main(String[] args) {
    Cliente c1 = new Cliente("Juliana");

    c1.adicionaAluguel(new Aluguel(new Fita("O Exorcista                    ", Fita.NORMAL), 3));
    c1.adicionaAluguel(new Aluguel(new Fita("Men in Black                   ", Fita.NORMAL), 2));
    c1.adicionaAluguel(new Aluguel(new Fita("Jurassic Park III              ", Fita.LANÇAMENTO), 3));
    c1.adicionaAluguel(new Aluguel(new Fita("Planeta dos Macacos            ", Fita.LANÇAMENTO), 4));
    c1.adicionaAluguel(new Aluguel(new Fita("Pateta no Planeta dos Macacos ", Fita.INFANTIL), 10));
    c1.adicionaAluguel(new Aluguel(new Fita("O Rei Leao                     ", Fita.INFANTIL), 30));

    System.out.println(c1.extrato());
  }
}
```

O que você acha desse código?

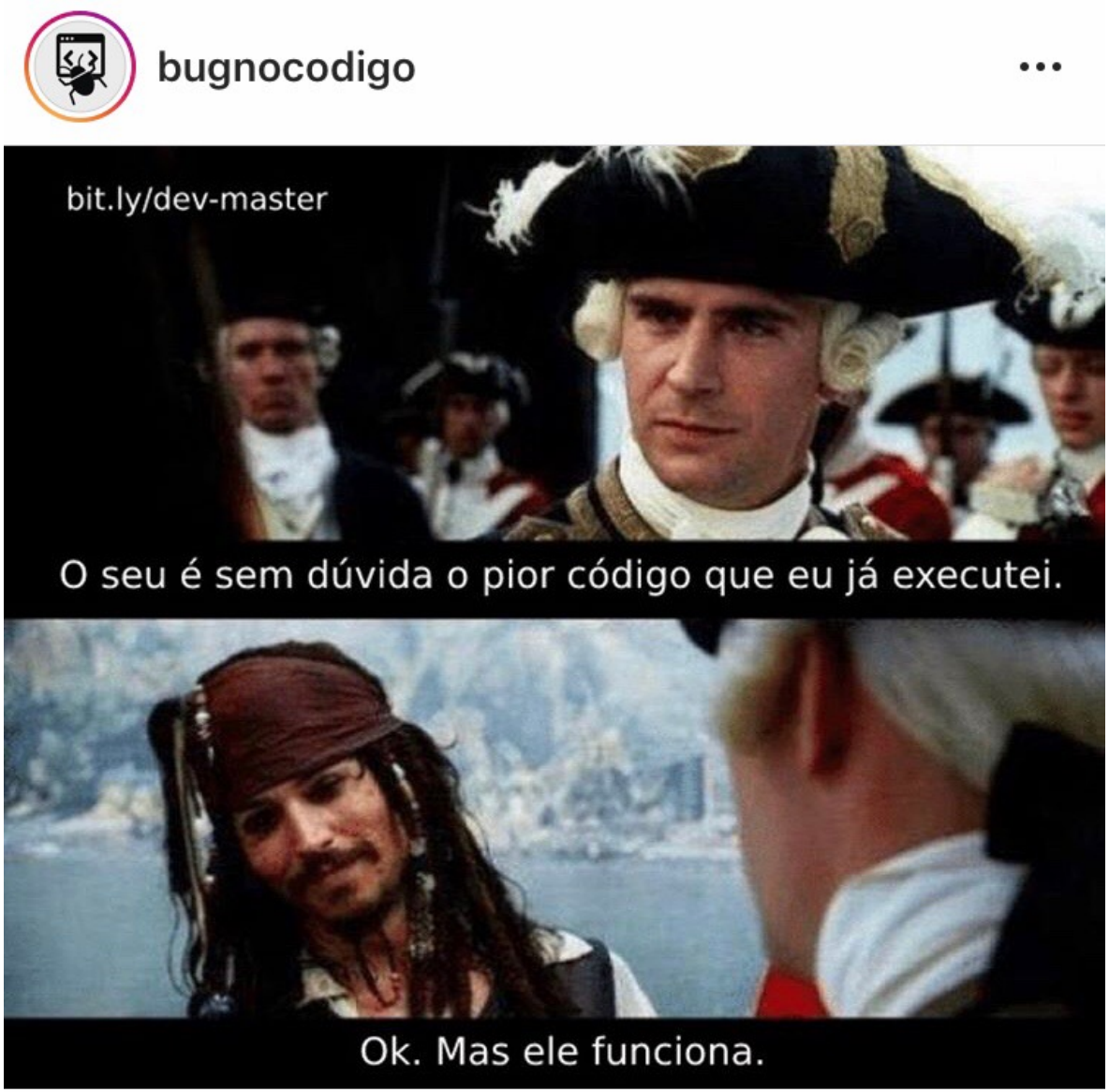
Mau Cheiro

- Como programador, você precisa treinar seu nariz para detectar "mau cheiro" (***bad smells***) em código

- O programa não está bem projetado e não é "orientado a objeto”

Mau Cheiro

- O mau cheiro que indica isso é:
  - O método extrato() é muito grande e faz tudo sozinho
  - Não há responsabilidades assumidas pelas classes Fita e Aluguel


bugnocodigo
bit.ly/dev-master
O seu é sem dúvida o pior código que eu já executei.
Ok. Mas ele funciona.

Mas o que importa se o programa funciona?

- Código ruim é difícil de alterar
- Se é difícil, então bugs serão introduzidos

Mas o que importa se o programa funciona?

- Exemplos:
  - A classificação em 3 tipos de fitas vai mudar, mas os donos da locadora não sabem exatamente o que querem ainda e você pode ter certeza que haverá várias mudanças ao longo do tempo
  - Nosso código está pronto para lidar facilmente com um novo esquema de classificação de fitas?
    - **Não**
  - Nosso código está pronto para lidar facilmente com um novo esquema de pontos de alugador frequente?
    - **Não**